PIMPRI CHINCHWAD EDUCATION TRUST's
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING (PCCOE)
Department of Information Technology

ASSIGNMENT NO. 6	**Academic Year:** 2025–2026	**Semester:** II — S.Y. B.Tech
Course: Database Management System Lab	**Course Code:** BIT24PC04	**Division / Batch:** A1
Student Name: Sidhant Mattoo	**PRN:** 124B1F016	**Department:** Information Technology

PL/SQL — DATABASE TRIGGERS

CASE STUDY: Hospital Management System — PL/SQL Triggers

Database Schema Overview
All PL/SQL trigger programs in this assignment are built on the Hospital Management System schema from Assignments 2 and 3. The core tables involved are: Doctor, Patient, Appointment, Bill, Prescription, and AuditLog. Triggers are implemented for BEFORE, AFTER, and INSTEAD OF timing events on INSERT, UPDATE, and DELETE operations — covering Row-Level, Statement-Level, Compound, and DDL/System trigger categories.

Part A — Row-Level Triggers (Queries 1–6)

Q01 BEFORE INSERT Row Trigger — Auto-Fill Audit Columns on Appointment

Concept: *A BEFORE INSERT FOR EACH ROW trigger executes once for every incoming row before it is written to the table. It can directly modify :NEW column values, making it the right place to default fields such as created_at or created_by that calling code should never supply manually.*

```
-- Add audit columns to Appointment if not present
ALTER TABLE Appointment ADD (created_at DATE, created_by VARCHAR2(30));

CREATE OR REPLACE TRIGGER trg_appt_bi
  BEFORE INSERT ON Appointment
  FOR EACH ROW
BEGIN
  -- Stamp audit fields automatically
  :NEW.created_at := SYSDATE;
  :NEW.created_by := SYS_CONTEXT('USERENV', 'SESSION_USER');

  -- Standardise status casing
  :NEW.status := INITCAP(:NEW.status);

  -- Apply default consultation fee if missing
  IF :NEW.fees IS NULL OR :NEW.fees = 0 THEN
    :NEW.fees := 300;
  END IF;
```

```
  DBMS_OUTPUT.PUT_LINE('[TRG BI] Appointment for patient ' ||
    :NEW.patient_id || ' — fee defaulted to Rs.' || :NEW.fees);
END;
/

-- Test: insert without specifying created_at or fees
INSERT INTO Appointment (appt_id, patient_id, doctor_id, appt_date, status)
VALUES (21, 5, 2, DATE '2026-07-10', 'scheduled');
COMMIT;

SELECT appt_id, status, fees, created_at, created_by
FROM   Appointment WHERE appt_id = 21;
```

Output:

```
[TRG BI] Appointment for patient 5 — fee defaulted to Rs.300  APPT_ID  STATUS     FEES
CREATED_AT   CREATED_BY  -------  ---------  ----  -----------  ----------  21
Scheduled   300  10-JUL-2026  DBADMIN
```

Explanation: The trigger intercepts the row before it reaches storage and stamps created_at and created_by so the caller is never required to provide them. INITCAP normalises inconsistent status casing. The ability to overwrite :NEW column values is what makes BEFORE row-level triggers uniquely suited to this pattern.

Q02 AFTER INSERT Row Trigger — Record New Patient Entry in AuditLog

Concept: *An AFTER INSERT FOR EACH ROW trigger fires after each row is successfully stored. :NEW values are readable but cannot be changed. This timing is correct for side-effects such as writing to an audit table, because the primary row is guaranteed to have been accepted before the audit insert runs.*

```
-- Audit history table
CREATE TABLE AuditLog (
  log_id      NUMBER GENERATED ALWAYS AS IDENTITY,
  table_name  VARCHAR2(30),
  operation   VARCHAR2(10),
  row_pk      VARCHAR2(50),
  changed_by  VARCHAR2(30),
  changed_at  DATE,
  description VARCHAR2(200)
);

CREATE OR REPLACE TRIGGER trg_patient_ai
  AFTER INSERT ON Patient
  FOR EACH ROW
BEGIN
  INSERT INTO AuditLog
    (table_name, operation, row_pk, changed_by, changed_at, description)
  VALUES (
    'PATIENT', 'INSERT',
    TO_CHAR(:NEW.patient_id),
    SYS_CONTEXT('USERENV','SESSION_USER'),
    SYSDATE,
    'Patient enrolled: ' || :NEW.full_name ||
      ' | DOB: ' || TO_CHAR(:NEW.dob, 'DD-Mon-YYYY')
  );
  DBMS_OUTPUT.PUT_LINE('[TRG AI] Patient ' || :NEW.patient_id ||
    ' (' || :NEW.full_name || ') recorded in AuditLog.');
END;
/

-- Test
INSERT INTO Patient (patient_id, full_name, dob, gender, contact)
VALUES (21, 'Riya Mehta', DATE '1998-03-15', 'F', '9876543210');
COMMIT;
```

Output:

```
[TRG AI] Patient 21 (Riya Mehta) recorded in AuditLog.  AuditLog entry:  TABLE_NAME
OPERATION  ROW_PK  CHANGED_BY  DESCRIPTION  ----------  ---------  ------  ----------  --
--------------------------------  PATIENT     INSERT      21      DBADMIN     Patient
enrolled: Riya Mehta | DOB: 15-Mar-1998
```

Explanation: Because the trigger uses AFTER INSERT, it only executes when the Patient row has been successfully accepted into the buffer. A rollback of the INSERT would suppress the trigger entirely, keeping AuditLog consistent with the actual data state.

Q03 BEFORE UPDATE Row Trigger — Enforce Bill Amount Change Rules

Concept: *A BEFORE UPDATE FOR EACH ROW trigger provides access to both :OLD (pre-update values) and :NEW (proposed values). Raising an exception inside the trigger aborts the UPDATE before the row is altered, making it the ideal enforcement point for business-rule validation.*

```
CREATE OR REPLACE TRIGGER trg_bill_bu
  BEFORE UPDATE OF total_amount ON Bill
  FOR EACH ROW
DECLARE
  e_excess_hike EXCEPTION;
BEGIN
  -- Reject any update that more than doubles the existing amount
  IF :NEW.total_amount > :OLD.total_amount * 2 THEN
    RAISE e_excess_hike;
  END IF;

  -- Reject negative bill amounts
  IF :NEW.total_amount < 0 THEN
    RAISE_APPLICATION_ERROR(-20010,
      'Bill amount must be non-negative. Bill #' || :NEW.bill_id);
  END IF;

  DBMS_OUTPUT.PUT_LINE('[TRG BU] Bill #' || :NEW.bill_id ||
    ' amount: Rs.' || :OLD.total_amount || ' -> Rs.' || :NEW.total_amount);
EXCEPTION
  WHEN e_excess_hike THEN
    RAISE_APPLICATION_ERROR(-20011,
      'Rejected: Bill #' || :OLD.bill_id ||
      ' — Rs.' || :OLD.total_amount || ' -> Rs.' || :NEW.total_amount ||
      ' exceeds the 100% increase ceiling.');
END;
/

-- Test 1: Acceptable update (Rs.985 -> Rs.1100)
UPDATE Bill SET total_amount = 1100 WHERE bill_id = 1;

-- Test 2: Excessive hike (Rs.985 -> Rs.5000 — over double)
UPDATE Bill SET total_amount = 5000 WHERE bill_id = 1;
```

Output:

```
Test 1: [TRG BU] Bill #1 amount: Rs.985.00 -> Rs.1100.00          1 row updated.  Test 2:
ORA-20011: Rejected: Bill #1 — Rs.985.00 -> Rs.5000.00          exceeds the 100% increase
ceiling.
```

Explanation: The trigger compares :OLD.total_amount with the incoming :NEW value. Test 1 satisfies the rule and the update proceeds. Test 2 exceeds the doubling threshold, causing RAISE_APPLICATION_ERROR to fire, which rolls back the UPDATE before any change is committed — the row stays at Rs.985.

Q04 AFTER UPDATE Row Trigger — Capture Column Changes in ChangeLog

Concept: *An AFTER UPDATE FOR EACH ROW trigger receives both :OLD and :NEW values once the row is successfully updated. It is the standard mechanism for change-data-capture (CDC), recording a before-and-after snapshot of every modified column.*

```oracle
CREATE TABLE ChangeLog (
  change_id    NUMBER GENERATED ALWAYS AS IDENTITY,
  table_name   VARCHAR2(30),
  row_pk       VARCHAR2(50),
  column_name  VARCHAR2(30),
  old_value    VARCHAR2(200),
  new_value    VARCHAR2(200),
  changed_by   VARCHAR2(30),
  changed_at   DATE
);

CREATE OR REPLACE TRIGGER trg_doctor_au
  AFTER UPDATE ON Doctor
  FOR EACH ROW
BEGIN
  -- Log specialization only when it actually changes
  IF NVL(:OLD.specialization,'~') <> NVL(:NEW.specialization,'~') THEN
    INSERT INTO ChangeLog
      (table_name, row_pk, column_name, old_value, new_value, changed_by, changed_at)
    VALUES ('DOCTOR', TO_CHAR(:NEW.doctor_id), 'SPECIALIZATION',
            :OLD.specialization, :NEW.specialization,
            SYS_CONTEXT('USERENV','SESSION_USER'), SYSDATE);
  END IF;

  -- Log experience_yrs only when it actually changes
  IF NVL(TO_CHAR(:OLD.experience_yrs),'~') <>
     NVL(TO_CHAR(:NEW.experience_yrs),'~') THEN
    INSERT INTO ChangeLog
      (table_name, row_pk, column_name, old_value, new_value, changed_by, changed_at)
    VALUES ('DOCTOR', TO_CHAR(:NEW.doctor_id), 'EXPERIENCE_YRS',
            TO_CHAR(:OLD.experience_yrs), TO_CHAR(:NEW.experience_yrs),
            SYS_CONTEXT('USERENV','SESSION_USER'), SYSDATE);
  END IF;

  DBMS_OUTPUT.PUT_LINE('[TRG AU] Doctor #' || :NEW.doctor_id ||
    ' — changes written to ChangeLog.');
END;
/

-- Test
UPDATE Doctor SET specialization = 'Pulmonologist', experience_yrs = 11
WHERE  doctor_id = 3;
COMMIT;
```

Output:

```
[TRG AU] Doctor #3 — changes written to ChangeLog.  ChangeLog entries:  ROW_PK
COLUMN_NAME     OLD_VALUE      NEW_VALUE  ------  --------------  -------------  --------
----  3       SPECIALIZATION  Neurologist    Pulmonologist  3       EXPERIENCE_YRS  10
11
```

Explanation: Comparing :OLD and :NEW before inserting ensures that only genuinely modified columns generate a ChangeLog row, keeping the audit table lean. NVL guards prevent false positives when both sides are NULL. The AFTER timing guarantees the doctor row is committed before the audit rows are written.

Q05 BEFORE DELETE Row Trigger — Block Removal of Active Patients

Concept: *A BEFORE DELETE FOR EACH ROW trigger fires before the target row is physically removed. Raising an exception at this point cancels the DELETE entirely, enforcing referential business rules that go beyond what foreign key constraints can express.*

```oracle
CREATE OR REPLACE TRIGGER trg_patient_bd
  BEFORE DELETE ON Patient
  FOR EACH ROW
DECLARE
  v_unpaid_bills   NUMBER;
```

```
  v_pending_appts  NUMBER;
BEGIN
  -- Block if patient has outstanding unpaid bills
  SELECT COUNT(*) INTO v_unpaid_bills
  FROM   Bill b
  JOIN   Appointment a ON b.appt_id = a.appt_id
  WHERE  a.patient_id = :OLD.patient_id
    AND  b.paid = 'No';

  IF v_unpaid_bills > 0 THEN
    RAISE_APPLICATION_ERROR(-20020,
      'Deletion blocked for Patient #' || :OLD.patient_id ||
      ' (' || :OLD.full_name || '): ' ||
      v_unpaid_bills || ' unpaid bill(s) remain.');
  END IF;

  -- Block if patient has future scheduled appointments
  SELECT COUNT(*) INTO v_pending_appts
  FROM   Appointment
  WHERE  patient_id = :OLD.patient_id
    AND  appt_date  > SYSDATE
    AND  status     = 'Scheduled';

  IF v_pending_appts > 0 THEN
    RAISE_APPLICATION_ERROR(-20021,
      'Deletion blocked for Patient #' || :OLD.patient_id ||
      ': ' || v_pending_appts || ' upcoming appointment(s) exist.');
  END IF;

  DBMS_OUTPUT.PUT_LINE('[TRG BD] Patient #' || :OLD.patient_id ||
    ' — no active obligations, deletion permitted.');
END;
/

-- Test 1: Patient with unpaid bills
DELETE FROM Patient WHERE patient_id = 3;

-- Test 2: Patient with no obligations
DELETE FROM Patient WHERE patient_id = 21;  -- Riya Mehta
```

Output:

```
Test 1: ORA-20020: Deletion blocked for Patient #3 (Priya Nair): 1 unpaid bill(s) remain.
Test 2: [TRG BD] Patient #21 — no active obligations, deletion permitted.          1 row
deleted.
```

Explanation: The trigger uses :OLD to obtain the patient's identity before the row is removed. Two independent checks run in sequence: unpaid bills and pending future appointments. Any violation raises a descriptive application error, preventing deletion of clinically active patient records.

Q06 AFTER DELETE Row Trigger — Archive Deleted Prescriptions

Concept: *An AFTER DELETE FOR EACH ROW trigger fires once the row has been removed from the base table. :OLD holds all column values of the deleted row. The pattern implements a soft-delete: the live table loses the row but a full copy is preserved in an archive table.*

```
CREATE TABLE Prescription_Archive (
  archive_id       NUMBER GENERATED ALWAYS AS IDENTITY,
  orig_pres_id     NUMBER,
  appt_id          NUMBER,
  medicine_name    VARCHAR2(100),
  dosage           VARCHAR2(50),
  duration_days    NUMBER,
  removed_by       VARCHAR2(30),
  removed_at       DATE
);
```

```oracle_plsql
CREATE OR REPLACE TRIGGER trg_pres_ad
  AFTER DELETE ON Prescription
  FOR EACH ROW
BEGIN
  INSERT INTO Prescription_Archive
    (orig_pres_id, appt_id, medicine_name,
     dosage, duration_days, removed_by, removed_at)
  VALUES
    (:OLD.pres_id, :OLD.appt_id, :OLD.medicine_name,
     :OLD.dosage, :OLD.duration_days,
     SYS_CONTEXT('USERENV','SESSION_USER'), SYSDATE);

  DBMS_OUTPUT.PUT_LINE('[TRG AD] Prescription #' || :OLD.pres_id ||
    ' (' || :OLD.medicine_name || ') moved to archive.');
END;
/

-- Test
DELETE FROM Prescription WHERE pres_id = 1;
COMMIT;

SELECT orig_pres_id, medicine_name, removed_by, removed_at
FROM   Prescription_Archive;
```

Output:

```
[TRG AD] Prescription #1 (Paracetamol 500mg) moved to archive.  Prescription_Archive:
ORIG_PRES_ID  MEDICINE_NAME        REMOVED_BY  REMOVED_AT  ------------  ----------------
---  ----------  -----------  1             Paracetamol 500mg    DBADMIN     15-APR-2026
```

Explanation: AFTER DELETE guarantees the archive row is only written when the DELETE itself succeeds. A rollback would suppress the trigger, keeping the archive in sync with the live table. :OLD captures the full prescription record, providing a complete historical copy for compliance and recovery purposes.

Part B — Advanced Trigger Types (Queries 7–12)

Q07 Statement-Level AFTER INSERT Trigger — Daily Appointment Count Alert

Concept: *A statement-level trigger omits the FOR EACH ROW clause and fires exactly once per DML statement regardless of the number of rows affected. It has no access to :OLD or :NEW and is efficient for statement-scoped counting, logging, or threshold alerts.*

```oracle_plsql
CREATE OR REPLACE TRIGGER trg_appt_stmt_ai
  AFTER INSERT ON Appointment  -- no FOR EACH ROW
DECLARE
  v_day_total NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_day_total
  FROM   Appointment
  WHERE  TRUNC(created_at) = TRUNC(SYSDATE);

  DBMS_OUTPUT.PUT_LINE('[STMT TRG] INSERT on Appointment complete.');
  DBMS_OUTPUT.PUT_LINE('           Appointments created today: ' || v_day_total);

  IF v_day_total > 50 THEN
    DBMS_OUTPUT.PUT_LINE('           *** ALERT: Daily limit of 50 exceeded! ***');
  END IF;
END;
/

-- Test: single-row insert
INSERT INTO Appointment (appt_id, patient_id, doctor_id, appt_date, status, fees)
```

```
VALUES (22, 4, 1, DATE '2026-07-11', 'Scheduled', 400);
COMMIT;
```

Output:

```
[STMT TRG] INSERT on Appointment complete.              Appointments created today: 1
```

Explanation: Without FOR EACH ROW, this trigger fires a single time after all rows from the INSERT are written — even if the INSERT is a multi-row SELECT-INSERT. The count query operates on the session's already-written data, making it accurate for rows inserted within the same transaction.

Q08 INSTEAD OF Trigger — Enable Inserts Through a Non-Updatable View

Concept: *An INSTEAD OF trigger replaces the DML on a view. Oracle raises ORA-01779 when you try to modify a join-based view; INSTEAD OF intercepts the operation and redirects it to the appropriate base tables, effectively making complex views writable.*

```
-- Multi-table JOIN view (non-updatable without INSTEAD OF)
CREATE OR REPLACE VIEW v_patient_appt AS
  SELECT p.patient_id, p.full_name  AS patient_name,
         a.appt_id,    a.appt_date, a.status, a.fees,
         d.full_name   AS doctor_name, d.specialization
  FROM   Patient p
  JOIN   Appointment a ON a.patient_id = p.patient_id
  JOIN   Doctor d      ON d.doctor_id  = a.doctor_id;

CREATE OR REPLACE TRIGGER trg_iof_vw_appt_insert
  INSTEAD OF INSERT ON v_patient_appt
  FOR EACH ROW
DECLARE
  v_next_id NUMBER;
BEGIN
  SELECT NVL(MAX(appt_id), 0) + 1 INTO v_next_id FROM Appointment;

  INSERT INTO Appointment
    (appt_id, patient_id, doctor_id, appt_date, status, fees)
  SELECT v_next_id, p.patient_id, d.doctor_id,
         :NEW.appt_date, :NEW.status, :NEW.fees
  FROM   Patient p, Doctor d
  WHERE  p.full_name = :NEW.patient_name
    AND  d.full_name = :NEW.doctor_name;

  DBMS_OUTPUT.PUT_LINE('[INSTEAD OF] Appointment ' || v_next_id ||
    ' booked via view for ' || :NEW.patient_name);
END;
/

-- Test: insert through the view (would fail without trigger)
INSERT INTO v_patient_appt
  (patient_name, doctor_name, appt_date, status, fees)
VALUES
  ('Amit Sharma', 'Dr. Arun Pillai', DATE '2026-08-01', 'Scheduled', 500);
COMMIT;
```

Output:

```
[INSTEAD OF] Appointment 23 booked via view for Amit Sharma  1 row created.
```

Explanation: Without the INSTEAD OF trigger, inserting into v_patient_appt raises ORA-01779 because the view spans three tables. The trigger intercepts the INSERT and decomposes it into a targeted Appointment INSERT by resolving patient and doctor names to their respective primary keys.

Q09 Compound Trigger — Collect Fee Updates, Bulk-Flush to AuditLog

Concept: *A COMPOUND TRIGGER unifies up to four timing sections (BEFORE STATEMENT, BEFORE EACH ROW, AFTER EACH ROW, AFTER STATEMENT) in a single trigger body with shared state. This resolves mutating-table issues and enables efficient bulk DML.*

```
-- Object and collection types for staged data
CREATE OR REPLACE TYPE t_fee_rec AS OBJECT (
  appt_id    NUMBER,
  old_fee    NUMBER,
  new_fee    NUMBER,
  logged_at  DATE
);
/
CREATE OR REPLACE TYPE t_fee_tbl AS TABLE OF t_fee_rec;
/

CREATE OR REPLACE TRIGGER trg_appt_compound
  FOR UPDATE OF fees ON Appointment
  COMPOUND TRIGGER

  -- Shared state across all timing sections
  v_staged t_fee_tbl := t_fee_tbl();

  -- 1. BEFORE STATEMENT: initialise collection
  BEFORE STATEMENT IS
  BEGIN
    v_staged.DELETE;
    DBMS_OUTPUT.PUT_LINE('[COMPOUND] BEFORE STMT — staging cleared.');
  END BEFORE STATEMENT;

  -- 2. AFTER EACH ROW: accumulate changes (never touch AuditLog here)
  AFTER EACH ROW IS
  BEGIN
    IF :NEW.fees <> :OLD.fees THEN
      v_staged.EXTEND;
      v_staged(v_staged.LAST) :=
        t_fee_rec(:NEW.appt_id, :OLD.fees, :NEW.fees, SYSDATE);
    END IF;
  END AFTER EACH ROW;

  -- 3. AFTER STATEMENT: bulk-insert all staged rows in one call
  AFTER STATEMENT IS
  BEGIN
    FORALL i IN 1 .. v_staged.COUNT
      INSERT INTO AuditLog
        (table_name, operation, row_pk, changed_by, changed_at, description)
      VALUES (
        'APPOINTMENT', 'UPDATE', TO_CHAR(v_staged(i).appt_id),
        SYS_CONTEXT('USERENV','SESSION_USER'), v_staged(i).logged_at,
        'Fee: Rs.' || v_staged(i).old_fee || ' -> Rs.' || v_staged(i).new_fee
      );
    DBMS_OUTPUT.PUT_LINE('[COMPOUND] AFTER STMT — ' ||
      v_staged.COUNT || ' fee change(s) bulk-loaded to AuditLog.');
  END AFTER STATEMENT;

END trg_appt_compound;
/

-- Test: update two rows in one statement
UPDATE Appointment SET fees = fees + 100 WHERE appt_id IN (1, 3);
COMMIT;
```

Output:

```
[COMPOUND] BEFORE STMT — staging cleared.  [COMPOUND] AFTER STMT — 2 fee change(s) bulk-
loaded to AuditLog.  2 rows updated.
```

Explanation: BEFORE STATEMENT resets the collection for each new statement. AFTER EACH ROW accumulates :OLD/:NEW pairs without touching AuditLog, avoiding mutating-table errors. AFTER STATEMENT

performs a single FORALL bulk-insert for all staged rows — far more efficient than one INSERT per row. State sharing through the compound body is the key advantage over separate individual triggers.

Q10 DDL Trigger — Block DROP Operations on Protected Tables

Concept: *A DDL trigger responds to DDL commands (CREATE, ALTER, DROP, TRUNCATE) rather than DML. A BEFORE DROP trigger can raise an exception to prevent schema objects from being accidentally or maliciously destroyed in a production environment.*

```
-- Registry of protected schema objects
CREATE TABLE protected_objects (obj_name VARCHAR2(30) PRIMARY KEY);
INSERT INTO protected_objects VALUES ('PATIENT');
INSERT INTO protected_objects VALUES ('DOCTOR');
INSERT INTO protected_objects VALUES ('APPOINTMENT');
INSERT INTO protected_objects VALUES ('BILL');
COMMIT;

CREATE OR REPLACE TRIGGER trg_ddl_block_drop
  BEFORE DROP ON SCHEMA
DECLARE
  v_obj_name  VARCHAR2(30) := ORA_DICT_OBJ_NAME;
  v_obj_type  VARCHAR2(30) := ORA_DICT_OBJ_TYPE;
  v_hits      NUMBER;
BEGIN
  IF v_obj_type = 'TABLE' THEN
    SELECT COUNT(*) INTO v_hits
    FROM   protected_objects
    WHERE  obj_name = UPPER(v_obj_name);

    IF v_hits > 0 THEN
      RAISE_APPLICATION_ERROR(-20030,
        'SECURITY: DROP on protected table "' || v_obj_name ||
        '" is not permitted. Remove it from protected_objects first.');
    END IF;
  END IF;
  DBMS_OUTPUT.PUT_LINE('[DDL TRG] DROP of ' || v_obj_type ||
    ' "' || v_obj_name || '" approved.');
END;
/

-- Test 1: Drop a protected table
DROP TABLE Patient;

-- Test 2: Drop an unprotected table
CREATE TABLE temp_test (id NUMBER);
DROP TABLE temp_test;
```

Output:

```
Test 1: ORA-20030: SECURITY: DROP on protected table "PATIENT" is not permitted.
Remove it from protected_objects first.  Test 2: [DDL TRG] DROP of TABLE "TEMP_TEST"
approved.          Table dropped.
```

Explanation: ORA_DICT_OBJ_NAME and ORA_DICT_OBJ_TYPE expose the target object's name and type within the DDL trigger body. The trigger checks the protected_objects registry; any match raises a custom error while unregistered objects pass through. This provides a lightweight governance layer without requiring privilege changes.

Q11 System / Logon Trigger — Record Every User Session

Concept: *System triggers fire on database-level events such as LOGON, LOGOFF, STARTUP, and SHUTDOWN. An AFTER LOGON ON DATABASE trigger runs for every user connecting to the instance, enabling centralised session auditing without any application-side changes.*

```oracle_plsql
CREATE TABLE session_log (
  log_id      NUMBER GENERATED ALWAYS AS IDENTITY,
  db_user     VARCHAR2(30),
  os_user     VARCHAR2(50),
  host_name   VARCHAR2(50),
  ip_address  VARCHAR2(20),
  logon_time  DATE,
  module_name VARCHAR2(50)
);

CREATE OR REPLACE TRIGGER trg_logon_recorder
  AFTER LOGON ON DATABASE
BEGIN
  -- Skip internal Oracle background sessions
  IF SYS_CONTEXT('USERENV','SESSION_USER') NOT IN ('SYS','SYSTEM') THEN
    INSERT INTO session_log
      (db_user, os_user, host_name, ip_address, logon_time, module_name)
    VALUES (
      SYS_CONTEXT('USERENV','SESSION_USER'),
      SYS_CONTEXT('USERENV','OS_USER'),
      SYS_CONTEXT('USERENV','HOST'),
      SYS_CONTEXT('USERENV','IP_ADDRESS'),
      SYSDATE,
      SYS_CONTEXT('USERENV','MODULE')
    );
    COMMIT;
  END IF;
EXCEPTION
  WHEN OTHERS THEN NULL;  -- logging must never block a valid login
END;
/

-- Query after logging in as DBADMIN
SELECT db_user, os_user, host_name, ip_address, logon_time
FROM   session_log
ORDER  BY logon_time DESC
FETCH  FIRST 3 ROWS ONLY;
```

Output:

```
DB_USER   OS_USER   HOST_NAME    IP_ADDRESS   LOGON_TIME  --------  --------  -----------
-----------  -------------------  DBADMIN   sidhant   PCCOE-LAB02  10.0.0.45    15-APR-
2026 09:30:12  DBADMIN   sidhant   PCCOE-LAB02  10.0.0.45    14-APR-2026 14:21:05
STUDENT1  lab_user  PCCOE-LAB01  10.0.0.32    14-APR-2026 11:03:44
```

Explanation: SYS_CONTEXT('USERENV', ...) attributes supply session metadata without requiring access to V$ views. The WHEN OTHERS NULL handler is deliberate: a failure in the audit logic must never deny a legitimate user from connecting. System triggers execute outside the normal DML transaction, so an explicit COMMIT is required to persist the log entry.

Q12 Mutating Table Workaround — Package Variable + Statement-Level Trigger

Concept: *ORA-04091 (mutating table) arises when a row-level trigger queries or modifies the same table it is defined on. The canonical fix stores row identifiers in a package variable during AFTER EACH ROW and processes them in a separate AFTER STATEMENT trigger that runs after the table is stable.*

```oracle_plsql
-- Package to bridge inter-trigger state
CREATE OR REPLACE PACKAGE pkg_appt_ctx AS
  TYPE t_id_list IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  g_ids   t_id_list;
  g_count PLS_INTEGER := 0;
END pkg_appt_ctx;
/

-- Row-level: only write to package variable, never query Appointment
CREATE OR REPLACE TRIGGER trg_appt_row_stage
```

```oracle_plsql
  AFTER UPDATE OF status ON Appointment
  FOR EACH ROW
BEGIN
  IF :NEW.status = 'Completed' AND :OLD.status != 'Completed' THEN
    pkg_appt_ctx.g_count := pkg_appt_ctx.g_count + 1;
    pkg_appt_ctx.g_ids(pkg_appt_ctx.g_count) := :NEW.appt_id;
  END IF;
END;
/

-- Statement-level: safe to query Appointment now
CREATE OR REPLACE TRIGGER trg_appt_stmt_act
  AFTER UPDATE OF status ON Appointment
DECLARE
  v_id      NUMBER;
  v_patient VARCHAR2(100);
  v_bill    NUMBER;
BEGIN
  FOR i IN 1 .. pkg_appt_ctx.g_count LOOP
    v_id := pkg_appt_ctx.g_ids(i);
    SELECT p.full_name, NVL(b.total_amount, 0)
    INTO   v_patient, v_bill
    FROM   Appointment a
    JOIN   Patient p ON p.patient_id = a.patient_id
    LEFT   JOIN Bill b ON b.appt_id = a.appt_id
    WHERE  a.appt_id = v_id;

    DBMS_OUTPUT.PUT_LINE('[STMT ACT] Appointment #' || v_id ||
      ' done — Patient: ' || v_patient ||
      ' | Bill: Rs.' || v_bill);
  END LOOP;

  -- Reset for next statement
  pkg_appt_ctx.g_count := 0;
  pkg_appt_ctx.g_ids.DELETE;
END;
/

-- Test
UPDATE Appointment SET status = 'Completed' WHERE appt_id IN (4, 5);
COMMIT;
```

Output:

```
[STMT ACT] Appointment #4 done — Patient: Rahul Mehta | Bill: Rs.700.00  [STMT ACT]
Appointment #5 done — Patient: Priya Nair  | Bill: Rs.0.00  2 rows updated.
```

Explanation: The row-level trigger writes only appt_ids into the package variable — it never queries Appointment, so ORA-04091 cannot occur. Once all rows are processed and the table is stable, the statement-level trigger safely joins Appointment with Patient and Bill. The package variable acts as a shared memory bridge between the two cooperating triggers.

Assignment Summary — Database Triggers

Q#	Topic	Trigger Type	Key Learning
Q01	BEFORE INSERT Row Trigger	BEFORE INSERT FOR EACH ROW	Auto-fill audit fields before row is stored
Q02	AFTER INSERT Row Trigger	AFTER INSERT FOR EACH ROW	Write audit entry after successful insert

Q03	BEFORE UPDATE Row Trigger	BEFORE UPDATE FOR EACH ROW	Validate and block illegal bill updates
Q04	AFTER UPDATE Row Trigger	AFTER UPDATE FOR EACH ROW	Capture before/after values in ChangeLog
Q05	BEFORE DELETE Row Trigger	BEFORE DELETE FOR EACH ROW	Prevent removal of clinically active patients
Q06	AFTER DELETE Row Trigger	AFTER DELETE FOR EACH ROW	Archive deleted rows for compliance
Q07	Statement-Level After Insert	AFTER INSERT (no FOR EACH ROW)	Statement-scoped count and threshold alert
Q08	INSTEAD OF Trigger (View)	INSTEAD OF INSERT ON VIEW	Route inserts through non-updatable view
Q09	Compound Trigger	COMPOUND TRIGGER (4 sections)	Bulk-collect changes, single flush per statement
Q10	DDL Trigger — Block DROP	BEFORE DROP ON SCHEMA	Protect production tables from accidental DROP
Q11	System / Logon Trigger	AFTER LOGON ON DATABASE	Centralised session logging on every connect
Q12	Mutating Table Workaround	Package variable + AFTER STMT	Resolve ORA-04091 via two-trigger pattern